

사전 빌드 DB 설계서 — Pre-built Firmware Database

목표: 예시 프롬프트 125개를 사전 빌드하여 DB화하고, 학생이 예시를 선택하면 AI 코드 생성(~8초) + 빌드(~25초) 과정을 생략하여 **즉시 펌웨어를 전송한다.**

1. 현재 vs 개선 비교

1.1 현재 방식 (실시간 빌드)



1.2 개선 방식 (사전 빌드 DB)



1.3 효과

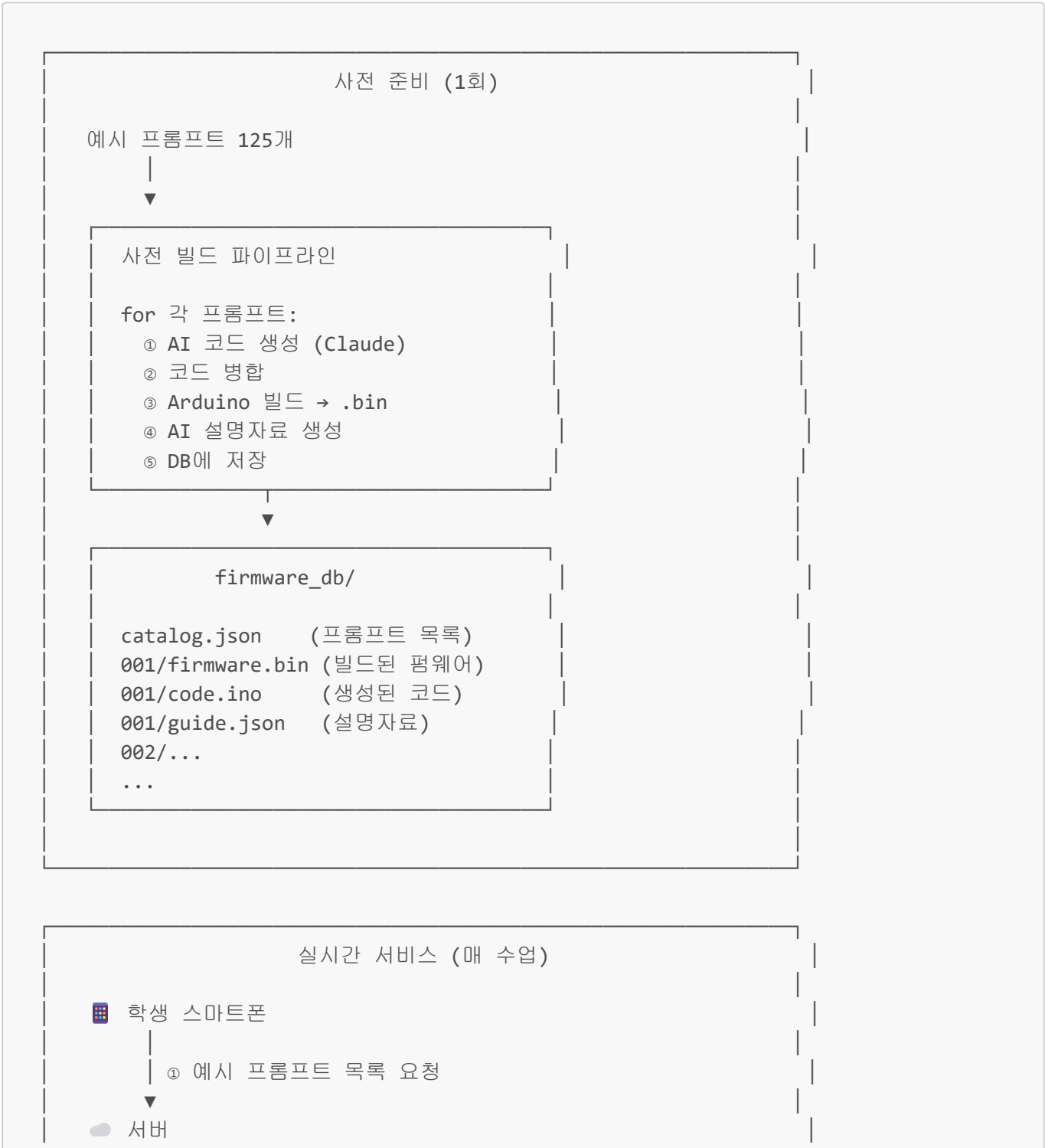
항목	현재	개선 후	절감
서버 처리 시간	~33초	~1초	-32초 (97% 단축)
E2E 소요 시간	~50초	~18초	-32초 (64% 단축)

항목	현재	개선 후	절감
AI API 비용	매번 과금	0원 (사전 생성)	100% 절감
서버 CPU 부하	매번 100%	거의 없음	빌드 대기열 제거
동시 사용자	1명씩 순차	제한 없음	30명 동시 가능

가장 큰 장점: 30명 학생이 동시에 예시를 선택해도 대기 없이 즉시 응답

2. 시스템 구조

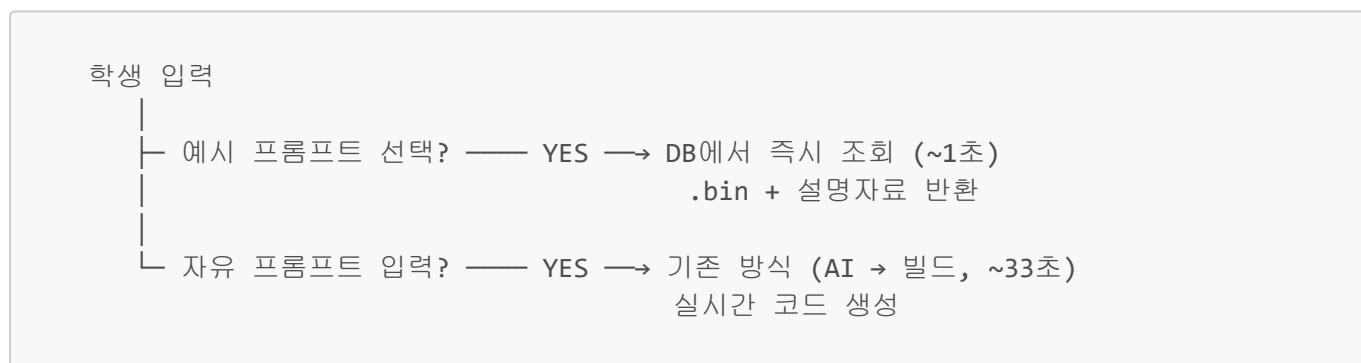
2.1 전체 아키텍처





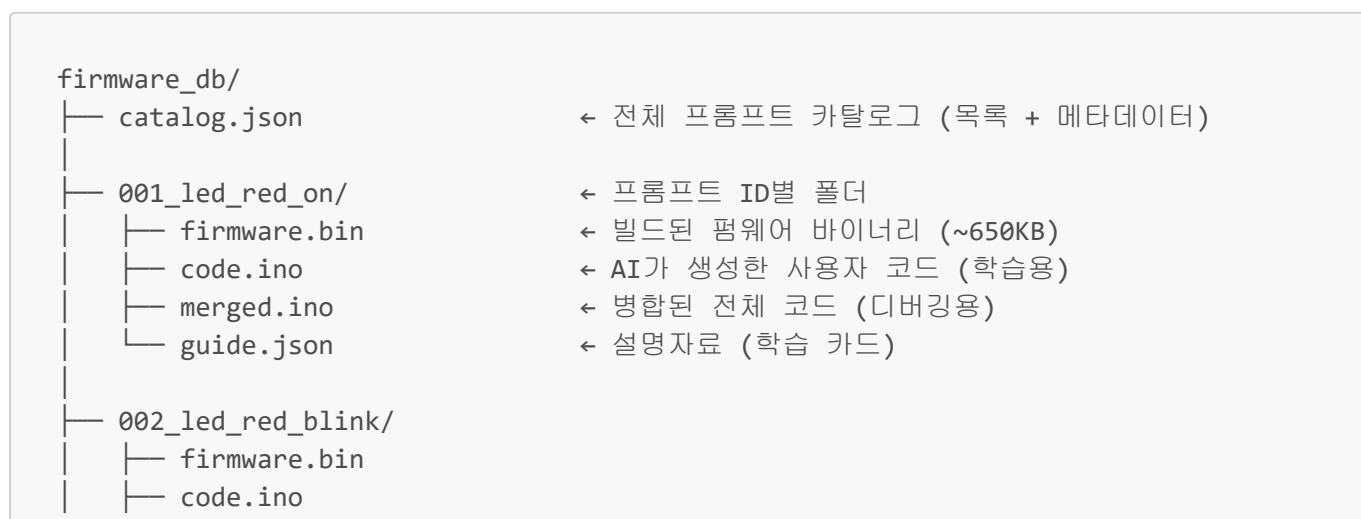
2.2 자유 프롬프트와의 병행

사전 빌드 DB는 **예시 프롬프트 전용**입니다. 학생이 자유 프롬프트를 입력하면 **기존 실시간 빌드** 방식으로 동작합니다.



3. DB 구조

3.1 디렉토리 구조



```

├── merged.ino
├── guide.json
├── ...
└── 125_digital_pet/
    ├── firmware.bin
    ├── code.ino
    ├── merged.ino
    └── guide.json

```

3.2 catalog.json 구조

```

{
  "version": "1.0",
  "board": "ESP32-WROOM-32",
  "build_date": "2026-04-20",
  "total": 125,
  "categories": [
    {
      "id": "led_basic",
      "name": "LED 기초",
      "icon": "●",
      "level": 1,
      "items": [
        {
          "id": "001",
          "prompt": "빨간 LED 켜기",
          "difficulty": 1,
          "components": ["led_red"],
          "concepts": ["digitalWrite", "출력"],
          "firmware_size": 651280,
          "firmware_sha256": "a1b2c3...",
          "description": "빨간 LED를 계속 켜 놓습니다"
        },
        {
          "id": "002",
          "prompt": "빨간 LED 1초마다 깜빡이기",
          "difficulty": 1,
          "components": ["led_red"],
          "concepts": ["delay", "반복"],
          "firmware_size": 651344,
          "firmware_sha256": "d4e5f6...",
          "description": "빨간 LED가 1초 간격으로 깜빡입니다"
        }
      ]
    },
    {
      "id": "led_advanced",
      "name": "LED 응용",
      "icon": "🚦",

```

```

    "level": 1,
    "items": [ ... ]
  },
  {
    "id": "buzzer_beep",
    "name": "부저 소리",
    "icon": "🔔",
    "level": 1,
    "items": [ ... ]
  }
]
}

```

3.3 guide.json 구조 (설명자료)

각 프롬프트별 학습 자료입니다. 웹앱에서 코드 학습 카드 대신 표시됩니다.

```

{
  "id": "002",
  "prompt": "빨간 LED 1초마다 깜빡이기",
  "difficulty": 1,
  "result_description": "빨간 LED가 1초 간격으로 계속 깜빡입니다",

  "what_happens": [
    "빨간 LED(25번 핀)가 0.5초 동안 켜집니다",
    "0.5초 동안 꺼집니다",
    "이것을 계속 반복합니다"
  ],

  "concepts": [
    {
      "term": "digitalWrite",
      "explanation": "GPIO 핀에 전기 신호를 보내는 함수. LOW를 보내면 LED가 꺼지고, HIGH를 보내면 켜집니다"
    },
    {
      "term": "delay",
      "explanation": "잠깐 멈추는 함수. delay(500)은 0.5초(500밀리초) 동안 멈춥니다"
    },
    {
      "term": "while (1)",
      "explanation": "종괄호 안의 동작을 무한으로 반복합니다. LED가 계속 깜빡이는 이유입니다"
    },
    {
      "term": "xTaskCreate",
      "explanation": "별도의 작업(태스크)을 만드는 함수. LED 깜빡이기를 백그라운드에서 실행합니다"
    }
  ],
}

```

```

"code_highlights": [
  {
    "line": "digitalWrite(LED_RED, LOW);",
    "comment": "빨간 LED를 켭니다 (LOW = 켜짐)"
  },
  {
    "line": "delay(500);",
    "comment": "0.5초 기다립니다"
  },
  {
    "line": "digitalWrite(LED_RED, HIGH);",
    "comment": "빨간 LED를 끕니다 (HIGH = 꺼짐)"
  }
],

"try_next": [
  { "id": "006", "prompt": "빨간 LED 0.5초마다 깜빡이기", "reason": "속도를 바꿔보  
세요" },
  { "id": "009", "prompt": "빨간 LED 5번 깜빡이고 멈추기", "reason": "횟수 제한에  
도전" },
  { "id": "015", "prompt": "3색 LED 동시에 깜빡이기", "reason": "여러 LED로 확장"
}
],

"quiz": {
  "question": "LED를 0.5초 동안 켜려면 delay에 몇을 넣어야 할까요?",
  "options": ["50", "500", "5000", "0.5"],
  "answer": 1,
  "explanation": "delay는 밀리초(1/1000초) 단위입니다. 0.5초 = 500밀리초"
}
}

```

4. 사전 빌드 파이프라인

4.1 빌드 스크립트 동작 흐름

125개 프롬프트를 순차적으로 빌드하는 **일괄 빌드 스크립트**가 필요합니다.

prebuild.py 실행



catalog_source.json 읽기 (125개 프롬프트 목록)



for 각 프롬프트:

① AI 코드 생성

→ Claude API 호출 (기존 SYSTEM_PROMPT 사용)

→ code.ino 저장

- ② 코드 병합
→ base firmware + AI 코드 → merged.ino
- ③ Arduino-CLI 빌드
→ merged.ino → firmware.bin
→ 빌드 실패 시 재시도 (최대 3회)
- ④ 설명자료 생성
→ Claude API에 코드 + 프롬프트를 보내서
guide.json 형식의 학습 자료 생성
- ⑤ DB 저장
→ firmware_db/{id}/ 에 4개 파일 저장
→ catalog.json에 메타데이터 추가
- ⑥ 검증
→ .bin 파일 크기 확인 (400KB~800KB 정상)
→ SHA256 해시 기록

↓
catalog.json 최종 생성
빌드 리포트 출력 (성공/실패 목록)

4.2 예상 소요 시간

항목	프롬프트당	125개 전체
AI 코드 생성	~10초	~21분
Arduino 빌드	~30초	~63분
설명자료 생성	~15초	~31분
합계	~55초	~115분 (약 2시간)

사전 빌드는 **1회만** 수행하면 됩니다. 프롬프트를 추가/수정할 때만 해당 항목만 재빌드합니다.

4.3 디스크 용량

항목	프롬프트당	125개 전체
firmware.bin	~650KB	~79MB
code.ino	~2KB	~250KB
merged.ino	~15KB	~1.8MB
guide.json	~3KB	~375KB
합계	~670KB	~82MB

서버 디스크 여유: 45GB → 82MB는 전혀 문제없음

5. API 설계

5.1 신규 API 엔드포인트

기존 API는 그대로 유지하고, 사전 빌드용 API를 **추가**합니다.

엔드포인트	Method	용도
/api/v1/catalog	GET	전체 프롬프트 카탈로그 조회
/api/v1/catalog/{category}	GET	카테고리별 프롬프트 조회
/api/v1/prebuilt/{id}	GET	사전 빌드 .bin 다운로드
/api/v1/prebuilt/{id}/guide	GET	설명자료 조회
/api/v1/prebuilt/{id}/code	GET	생성된 코드 조회
/api/v1/prebuilt/search?q=...	GET	프롬프트 검색 (한국어)

5.2 API 흐름

학생 스마트폰

- 앱 시작 시
 - GET /api/v1/catalog
 - 125개 프롬프트 목록 수신 (카테고리별, 난이도별)
 - 웹앱에 카테고리별 예시 목록 표시
- 학생이 예시 선택 (예: "빨간 LED 깜빡이기" 터치)
 - GET /api/v1/prebuilt/002
 - firmware.bin 즉시 다운로드 (~650KB, ~1초)
 - GET /api/v1/prebuilt/002/guide
 - guide.json 수신 (설명자료)
- BLE OTA 전송
 - 기존과 동일 (~17초)
- 전송 완료 후
 - guide.json의 내용으로 학습 카드 표시
 - 퀴즈 표시
 - "다음 도전" 프롬프트 추천

6. 웹앱 UI 변경

6.1 홈 탭 변경사항

기존 예시 프롬프트 칩(5개) → 카테고리별 예시 목록으로 확장



6.2 예시 선택 후 화면

학생이 "빨간 LED 1초마다 깜빡이기" 터치



즉시 실행 화면

⚡ 빨간 LED 1초마다 깜빡이기

설명

빨간 LED가 0.5초 켜지고 0.5초 꺼지기를 반복합니다

무슨 일이 일어나나요?

1. 빨간 LED(25번 핀)가 0.5초 동안 켜집니다
2. 0.5초 동안 꺼집니다
3. 이것을 계속 반복합니다

[⚡ 지금 보드에 보내기] ← 터치하면 즉시 BLE 전송

- ▶ 코드 보기 (접기/펼치기)
- ▶ 학습 퀴즈 (접기/펼치기)

다음 도전!

- LED 0.5초마다 깜빡이기 (속도 변경)
- LED 5번 깜빡이고 멈추기 (횟수 제한)
- 3색 LED 동시에 깜빡이기 (확장)

6.3 학습 퀴즈 (guide.json 기반)

퀴즈

Q. LED를 0.5초 동안 켜려면 delay에 몇을 넣어야 할까요?

- [A. 50]
- [B. 500] ← 정답
- [C. 5000]
- [D. 0.5]

✅ 정답! delay는 밀리초(1/1000초) 단위입니다.
0.5초 = 500밀리초

7. 구현 단계 (로드맵)

Phase 1: DB 구축 (사전 빌드)

작업 내용:

- └─ catalog_source.json 작성 (125개 프롬프트 + 메타데이터)
- └─ prebuild.py 스크립트 개발
 - └─ Claude API 호출 → 코드 생성
 - └─ 코드 병합 + Arduino-CLI 빌드
 - └─ Claude API 호출 → 설명자료(guide.json) 생성
 - └─ 결과물 저장 + catalog.json 생성

- └─ 125개 일괄 빌드 실행 (~2시간)
- └─ 빌드 결과 검증 (성공률, .bin 크기, 동작 테스트)

Phase 2: 서버 API 추가

작업 내용:

- └─ build_server_cloud_arduino.py에 API 추가
 - └─ GET /api/v1/catalog
 - └─ GET /api/v1/prebuilt/{id}
 - └─ GET /api/v1/prebuilt/{id}/guide
 - └─ GET /api/v1/prebuilt/{id}/code
 - └─ GET /api/v1/prebuilt/search?q=...
- └─ firmware_db/ 디렉토리를 서버에 배포
- └─ 기존 API (generate, status, download)는 그대로 유지

Phase 3: 웹앱 UI 변경

작업 내용:

- └─ 홈 탭 UI 변경
 - └─ 카테고리별 예시 프롬프트 목록
 - └─ 예시 선택 → 즉시 다운로드 + BLE 전송
 - └─ 설명자료 카드 표시 (guide.json 기반)
 - └─ 퀴즈 표시
 - └─ "다음 도전" 추천
- └─ 자유 프롬프트 영역은 유지 (기존 방식 병행)
- └─ ⚡ 즉시 배지로 사전빌드/실시간 빌드 구분

Phase 4: 배포 + 테스트

작업 내용:

- └─ 서버 배포 (firmware_db/ 업로드)
- └─ 웹앱 업데이트
- └─ E2E 테스트 (예시 선택 → 다운로드 → BLE 전송 → 동작)
- └─ 교사 매뉴얼 업데이트

8. 운영 및 유지보수

8.1 프롬프트 추가/수정

새 프롬프트를 추가하거나 기존 프롬프트를 수정할 때:

1. catalog_source.json에 프롬프트 추가/수정
2. prebuild.py --only {id} 로 해당 항목만 재빌드

3. 서버에 `firmware_db/{id}/` 업로드
4. `catalog.json` 업데이트 배포

8.2 펌웨어 베이스 변경 시

`ble_ota_arduino.ino`(베이스 펌웨어)가 변경되면 **전체 재빌드**가 필요합니다.

1. `prebuild.py --all` 실행 (125개 전체 재빌드, ~2시간)
2. 서버에 `firmware_db/` 전체 재배포

8.3 보드 변경 대응

aiEspMini(ESP32-C3) 등 다른 보드를 추가할 경우:

```
firmware_db/
├── wroom/           ← ESP32-WROOM 전용
│   ├── catalog.json
│   └── 001/ ~ 125/
└── c3_mini/         ← ESP32-C3 SuperMini 전용
    ├── catalog.json
    └── 001/ ~ 125/
```

`catalog.json`에 **board** 필드를 추가하여 보드별 분기합니다. 웹앱의 ⚙️설정 탭에서 보드 선택 → 해당 보드의 catalog 로드.

9. 기대 효과

9.1 수업 효율

항목	현재	개선 후
예시 1개 실행 시간	~50초	~18초
30명 순차 빌드 시 마지막 학생 대기	~25분	0분 (동시 가능)
2시간 수업에서 실습 가능 횟수 (학생당)	~10회	~30회
AI API 비용 (수업당)	125회 × API 과금	0원 (예시 사용 시)

9.2 교육 품질

항목	현재	개선 후
학습 자료	코드 주석만 (자동 생성)	맞춤 설명 + 퀴즈 + 다음 도전 추천
난이도 체계	없음 (자유 입력)	★☆☆ ~ ★★★★★ 단계별 분류

항목	현재	개선 후
커리큘럼 연동	없음	마일스톤별 예시 그룹화

10. 검토 사항 및 의사결정 필요 항목

10.1 확정 사항

- ☒ 125개 예시 프롬프트를 사전 빌드
- ☒ .bin + 설명자료를 DB 형태로 저장
- ☒ 예시 선택 시 즉시 다운로드 (AI+빌드 생략)
- ☒ 자유 프롬프트는 기존 방식 유지 (병행)

10.2 결정 필요 항목

항목	선택지	권장	이유
DB 저장 형식	파일 시스템 vs SQLite	파일 시스템	단순, 디버깅 용이, 서버 배포 쉬움
설명자료 생성 시점	사전 빌드 시 vs 수동 작성	사전 빌드 시 AI 생성	125개 수동 작성 비현실적
카탈로그 캐싱	앱 시작 시 전체 로드 vs 카테고리별	전체 로드	catalog.json ~50KB, 1회 로드로 충분
ESP32-C3 동시 지원	지금 같이 vs 나중에	나중에	WROOM 먼저 완성 후 C3 확장
빌드 검증 방법	.bin 크기만 vs 실제 보드 테스트	크기 + 샘플 보드 테스트	전수 테스트 비현실적, 카테고리별 1~2개 실제 테스트

작성일: 2026-04-16 관련 문서: 빌드_파이프라인_상세분석_및_최적화.md, 예시_프롬프트_목록.md